

Testes de software

Como reduzir risco,
aprender mais
cedo e projetar
mudanças com
segurança

Engenharia de software ·
guia introdutório



Testar é
reduzir
risco, não
provar
perfeição

Evidência
suficiente para
decidir com
mais segurança.

Testes

Evidências sobre o comportamento do sistema em cenários definidos.

Objetivo

Encontrar falhas cedo, proteger decisões e tornar mudanças mais seguras.

O que um bom teste precisa responder

Uma regra clara transforma execução em evidência.

01 Comportamento

O que o sistema deve fazer?

02 Contexto

Em quais condições?

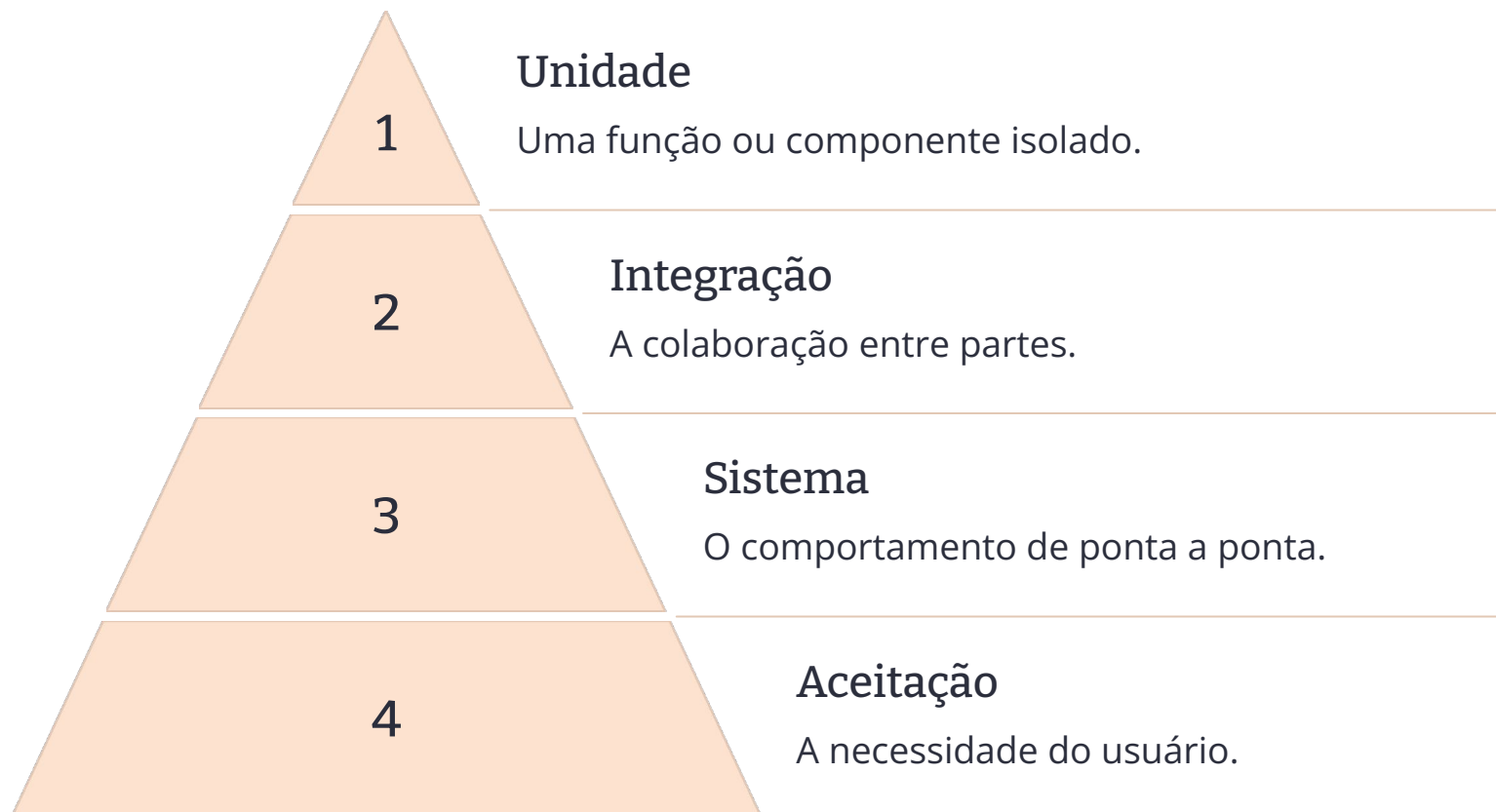
03 Resultado

Qual evidência mostra que funcionou?

Conectar cada teste a uma regra ou risco evita testes frágeis e sem propósito.

Os níveis de teste formam uma rede de proteção

Cada nível cobre um tipo diferente de risco.



Testes unitários priorizam feedback rápido

Uma decisão pequena, executada perto do código.

O que favorecem

Execução rápida, isolamento de uma decisão pequena e diagnóstico direto.

Devem ser legíveis e determinísticos: o mesmo cenário produz a mesma resposta.

O que não substituem

Mocks e stubs controlam dependências, mas não revelam se componentes reais colaboram corretamente.

Para isso, ainda são necessários testes de integração.

Integração e sistema revelam riscos de colaboração

Uma unidade pode funcionar sozinha e ainda falhar quando depende de contratos, dados ou serviços reais.



Qualidade começa antes do código

Risco descoberto cedo custa menos para corrigir.

01

Explicitar

Critérios de aceitação tornam o comportamento esperado claro.

03

Observar

Observabilidade mostra o que acontece quando o sistema está em uso.

02

Revisar

Revisão e análise estática encontram problemas antes da execução.

04

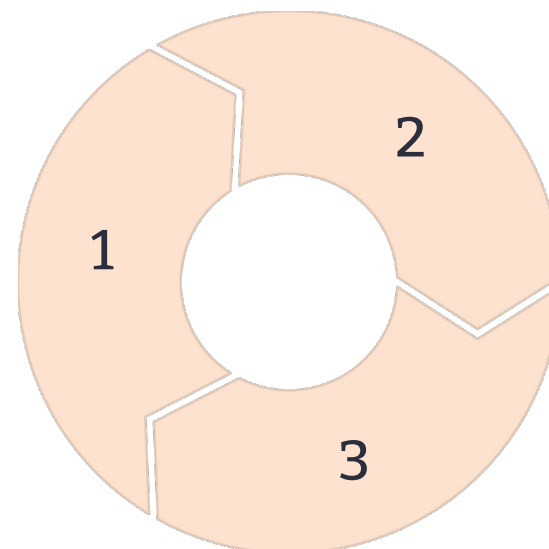
Testar

Testes reúnem evidências sobre os riscos que permanecem.

TDD transforma o teste em parte do design

O ciclo curto especifica, implementa, aprende e ajusta.

Red
Escreva um teste que
falha e exponha o
próximo
comportamento.



O teste guia o design ao tornar a próxima decisão explícita.

Green

Implemente o mínimo
necessário para fazê-lo
passar.

Refactor

Melhore a estrutura sem
mudar o comportamento
protegido.